



Informe Técnico Analítico

Materia: Paradigmas y Lenguajes

Integrantes - Grupo 14:

- Gaitán Pereyra Julio Emilio
- Gabaldo Zdanovicz Matías Nahuel
- Bonessi Luis María
- Rojas Ruth
- Frias Juan Gabriel.



Introducción

El presente Trabajo Práctico Integrador tuvo como objetivo desarrollar un Sistema de Semáforos Inteligentes utilizando el paradigma funcional en Common Lisp, aplicando conceptos de funciones puras, inmutabilidad y composición funcional.

Además de implementar la lógica principal del sistema, se investigó el ecosistema de herramientas disponible para Common Lisp mediante la integración de una librería externa, y se realizó un estudio comparativo con otro lenguaje funcional asignado por la cátedra, en nuestro caso “Clojure”.

El proyecto permitió comprender cómo modelar un problema del mundo real utilizando programación funcional, analizar el flujo de datos sin mutación de estado y comparar diferentes enfoques para resolver una misma problemática.

Fase 1 — Sistema de Semáforos Inteligentes

Restricciones de Diseño Aplicadas

Durante el desarrollo se respetaron las restricciones impuestas por la consigna:

1. Inmutabilidad Absoluta

No se utilizaron mecanismos de almacenamiento global mutable como:

- `defparameter`
- `defvar`
- `setq`
- `setf`

El estado del sistema se transmite exclusivamente mediante argumentos y valores de retorno.

2. Cero Bucles Imperativos

No se utilizaron estructuras imperativas como:

- `loop`
- `dolist`
- `dotimes`
- `while`

Las operaciones repetitivas fueron resueltas mediante funciones de orden superior y transformaciones funcionales.



Requerimiento 1: Estados de Transición

Se implementó la función “transición”, encargada de modelar los cambios de estado del semáforo.

En una primera iteración se trabajó con tres estados principales:

- en-rojo
- en-amarillo
- en-verde

Posteriormente se desarrolló una segunda iteración incorporando el estado:

- intermitente

Esto permitió representar de manera más realista las transiciones entre colores.

La función fue clasificada como:

```
;; =====  
;; FUNCIÓN: transicion  
;; NATURALEZA: Pura (Dado un color actual y uno nuevo, siempre retorna la misma acción)  
;; ESTRATEGIA: Condicional (Implementada mediante cond y comparaciones con equal)  
;; IMPACTO: No destructiva (Solo describe la transición, sin alterar datos externos)  
;; =====
```



Requerimiento 2: Temporizador Automático

Se implementó la función “timer”, encargada de determinar el estado actual del semáforo a partir de un timestamp en formato Unix (Epoch).

La lógica se basa en el uso de la operación: $(\text{mod timestamp } 216)$, que permite reutilizar indefinidamente el mismo ciclo temporal.

La 1ra iteración del proyecto contempla: Duración total del ciclo: 216 segundos

Estado	Duración
Rojo	90 segundos
Verde	120 segundos
Amarillo	6 segundos

En la 2da iteración se agregan al ciclo, las transiciones de intermitencia, por lo cual la duración total del ciclo ahora será de 225 segundos:

Estado	Duración
Rojo	90 segundos
Intermitente	3 segundos
Verde	120 segundos
Intermitente	3 segundos
Amarillo	6 segundos
Intermitente	3 segundos



La función fue clasificada como:

```
;; =====  
;; FUNCIÓN: timer  
;; NATURALEZA: Pura (Dado un timestamp, siempre retorna el mismo color del ciclo)  
;; ESTRATEGIA: Implementada mediante let y cond  
;; IMPACTO: No destructiva (Solo calcula el estado del semáforo, sin alterar datos externos)  
;; =====
```

Requerimiento 3: Sistema de Auditoría

Se implementó la función “registro-de-estados”, responsable de registrar los cambios de estado del semáforo.

Su finalidad es permitir auditorías posteriores y reconstruir eventos ocurridos durante la ejecución del sistema.

El registro se realiza mediante “format” y muestra:

- Fecha.
- Hora.
- Estado anterior.
- Estado nuevo.

Ejemplo: [2026-06-14 18:30:25] la luz ha cambiado de EN-ROJO a EN-VERDE

Esta función es la única clasificada como “impura”, ya que genera salida por pantalla.

```
;; =====  
;; FUNCIÓN: registro-de-estados  
;; NATURALEZA: Impura (Genera efectos externos al imprimir en consola)  
;; ESTRATEGIA: Implementada mediante format y local-time:format-timestring)  
;; IMPACTO: No destructiva (Solo registra cambios de estado, no altera datos internos)  
;; =====
```



Requerimiento 4: Análisis de Ciclos Semafóricos

Se desarrollaron las funciones:

- a) duracion-ciclo
- b) recomendacion-ciclo

Estas permiten calcular la cantidad de ciclos completos y emitir recomendaciones según las reglas de negocio establecidas.

La consigna establece que:

- Ciclos menores a 35 segundos son considerados demasiado cortos.
- Ciclos mayores a 150 segundos son considerados demasiado largos.
- Los ciclos comprendidos entre ambos valores son considerados óptimos.

En la 2da iteración, el único cambio realizado fue en la variable local de la función “duración-ciclo”, donde se agregó la cantidad de seg en total de intermitencia, alterando a sí mismo el valor final de lo que es un ciclo completo.

Las funciones fueron clasificadas como:

4-a:

```
;; =====  
;; FUNCIÓN: duracion-ciclo  
;; NATURALEZA: Pura (depende solo de los parámetros de entrada, no modifica estado)  
;; ESTRATEGIA: Uso de let para definir duración total del ciclo y floor para calcular ciclos completos  
;; IMPACTO: No destructiva (retorna lista con número de ciclos y recomendación, no altera variables globales)  
;; =====
```

4-b:

```
;; =====  
;; FUNCIÓN: recomendacion-ciclo  
;; NATURALEZA: Pura (depende solo del parámetro de entrada, no modifica estado)  
;; ESTRATEGIA: Condicional con verificación de tipo (integerp) y rangos  
;; IMPACTO: No destructiva (retorna lista con evaluación y recomendación, no altera variables globales)  
;; =====
```



Requerimiento 5: Planificación Temporal

Se implementó la función “ciclos-por-tiempo”. La misma recibe una cantidad de minutos, realiza la conversión a segundos y determina cuántos ciclos completos del semáforo pueden ejecutarse dentro de dicho período. Ejemplo:

Valor ingresado → (ciclos-por-tiempo 32)

Resultado → 8 ciclos completos

En la 2da iteración, en esta función al igual que en el requerimiento 4-a, se agregan los 9 seg de intermitencia, alterando la duración del ciclo completo.

La función fue clasificada como:

```
;; =====  
;; FUNCIÓN: ciclos-por-tiempo  
;; NATURALEZA: Pura (depende solo de los parámetros de entrada, no modifica estado)  
;; ESTRATEGIA: Conversión de minutos a segundos y división entera con floor  
;; IMPACTO: No destructiva (retorna número de ciclos completos, no altera variables globales)  
;; =====
```

Requerimiento 6: Informe de Distribución Temporal

Se desarrolló la función “distribucion_porcentual”. Para su implementación se utilizaron funciones de orden superior:

- Mapcar
- lambda

El objetivo es calcular qué porcentaje del tiempo total corresponde a cada estado del semáforo.

La función devuelve una lista con los porcentajes asociados a:

- Rojo.
- Amarillo.
- Verde.
- Intermitente.

Esta solución mantiene la filosofía funcional del proyecto al evitar modificaciones de estructuras existentes.



En la 2da iteración, además de agregar los 9 seg de intermitencia en la variable local, también lo agregamos en la lista que recibe mapcar, impactando así en el resultado de los porcentajes sobre lo que será la duración total de ciclos en una hora.

La función fue clasificada como:

```
;; =====  
;; FUNCIÓN: distribucion_porcentual  
;; NATURALEZA: Pura (Dado un ciclo fijo, siempre retorna los mismos porcentajes)  
;; ESTRATEGIA: Orden Superior (Implementada mediante let*, mapcar y lambda)  
;; IMPACTO: No destructiva (Solo calcula proporciones de tiempos de colores)  
;; =====
```

Requerimiento 7: Aseguramiento de la calidad

Para finalizar esta 1ra fase, se realizaron las pruebas correspondientes para comprobar el funcionamiento correcto de cada función, a continuación se muestran ejemplos teniendo en cuenta que pueden haber 3 caminos posibles: (normales, alternativos y casos de errores):

```
;;; ===== REQUERIMIENTO 1: transicion =====  
  
;;; Funcionamiento NORMAL:  
;;; valor ingresado -> (transicion 'en-rojo 'verde)  
;;; valor devuelto -> (EN-ROJO "Cambiar-a-verde")  
  
;;; valor ingresado -> (transicion 'en-amarillo 'rojo)  
;;; valor devuelto -> (EN-AMARILLO "Cambiar-a-rojo")  
  
;;; valor ingresado -> (transicion 'en-verde 'amarillo)  
;;; valor devuelto -> (EN-VERDE "Cambiar-a-amarillo")  
  
;;; Funcionamiento con camino ALTERNATIVO:  
;;; valor ingresado -> (transicion 'amarillo 'rojo)  
;;; valor devuelto -> (AMARILLO "Accion-por-defecto")  
  
;;; Caso de ejemplo de ERROR:  
;;; valor ingresado -> (transicion 'en-rojo)  
;;; valor devuelto -> *** - invalid number of arguments: 1
```




```
;;; ===== REQUERIMIENTO 2: timer =====

;;; Funcionamiento NORMAL:
;;; valor ingresado -> (timer 1000)
;;; valor devuelto -> EN-VERDE

;;; valor ingresado -> (timer 216)
;;; valor devuelto -> EN-ROJO

;;; Funcionamiento con camino ALTERNATIVO:
;;; valor ingresado -> (timer 95)
;;; valor devuelto -> EN-AMARILLO

;;; Caso de ejemplo de ERROR:
;;; valor ingresado -> (timer hola)
;;; valor devuelto -> *** - SYSTEM::READ-EVAL-PRINT: variable HOLA has no value
```

```
;;; ===== REQUERIMIENTO 3: registrar-cambio =====

;;; Funcionamiento NORMAL:
;;; valor ingresado -> (registrar-cambio 90 'en-rojo 'en-verde)
;;; valor devuelto -> Tiempo 90: la luz ha cambiado de EN-ROJO a EN-VERDE

;;; Funcionamiento con camino ALTERNATIVO:
;;; valor ingresado -> (registrar-cambio 10000 'en-rojo 'en-verde)
;;; valor devuelto -> Tiempo 10000: la luz ha cambiado de EN-ROJO a EN-VERDE

;;; valor ingresado -> (registrar-cambio 90.5 'en-rojo 'en-verde)
;;; valor devuelto -> Tiempo 90.5: la luz ha cambiado de EN-ROJO a EN-VERDE

;;; Caso de ejemplo de ERROR / LIMITE:
;;; valor ingresado -> (registrar-cambio -10 'en-rojo 'en-verde)
;;; valor devuelto -> Tiempo -10: la luz ha cambiado de EN-ROJO a EN-VERDE
;;; (no produce error, pero un timestamp negativo no tiene sentido en el dominio)

;;; valor ingresado -> (registrar-cambio 100 nil 'en-verde)
;;; valor devuelto -> Tiempo 100: la luz ha cambiado de NIL a EN-VERDE
;;; (no produce error, pero NIL como "color anterior" es semánticamente inválido)
```



```
;;; ===== REQUERIMIENTO 4: duracion-ciclo / recomendacion-ciclo =====  
  
;;; Funcionamiento NORMAL:  
;;; valor ingresado -> (duracion-ciclo 32747)  
;;; valor devuelto  -> (151 ("Rango NO-optimo:" CICLO-LARGO "recomendacion:" AUMENTAR-DURACION))  
  
;;; Funcionamiento con camino ALTERNATIVO:  
;;; valor ingresado -> (duracion-ciclo 3172.3)  
;;; valor devuelto  -> (14 NIL)  
  
;;; Caso de ejemplo de ERROR:  
;;; valor ingresado -> (duracion-ciclo 'treintaycuatro)  
;;; valor devuelto  -> *** - FLOOR: TREINTAYCUATRO is not a real number
```

```
;;; ===== REQUERIMIENTO 5: ciclos-por-tiempo =====  
  
;;; Funcionamiento NORMAL:  
;;; valor ingresado -> (ciclos-por-tiempo 32)  
;;; valor devuelto  -> 8  
  
;;; Funcionamiento con camino ALTERNATIVO:  
;;; valor ingresado -> (ciclos-por-tiempo 50.05)  
;;; valor devuelto  -> 13  
  
;;; Caso de ejemplo de ERROR:  
;;; valor ingresado -> (ciclos-por-tiempo 'trescientostreinta)  
;;; valor devuelto  -> *** - *: TRESCIENTOSTREINTA is not a number
```

```
;;; ===== REQUERIMIENTO 6: distribucion_porcentual =====  
  
;;; Funcionamiento NORMAL:  
;;; valor ingresado -> (distribucion_porcentual 90 6 120)  
;;; valor devuelto  -> ((EN-ROJO 41.666664) (EN-AMARILLO 2.7777777) (EN-VERDE 55.55556))  
  
;;; Funcionamiento con camino ALTERNATIVO:  
;;; valor ingresado -> (distribucion_porcentual 72 72 72)  
;;; valor devuelto  -> ((EN-ROJO 33.333332) (EN-AMARILLO 33.333332) (EN-VERDE 33.333332))  
  
;;; Caso de ejemplo de ERROR:  
;;; valor ingresado -> (distribucion_porcentual 0 0 0)  
;;; valor devuelto  -> *** - DIVISION-BY-ZERO
```



Fase 2 — Autonomía y Ecosistema

Quicklisp

Para la segunda fase se investigó el gestor de paquetes “Quicklisp”, utilizado ampliamente en el ecosistema Common Lisp.

Quicklisp permite:

- Instalar librerías externas.
- Gestionar dependencias.
- Cargar paquetes automáticamente.

Su funcionamiento es similar al de gestores de paquetes presentes en otros lenguajes modernos.

Librería Seleccionada: Local-Time

La librería elegida fue “Local-Time”. Su incorporación permitió mejorar el sistema de auditoría del requerimiento 3. Sin esta librería los tiempos se mostrarían únicamente como números Unix Epoch, lo que resulta poco práctico para un usuario humano. Por ejemplo:

Valor recibido → 1749931200

Se muestra como → 2026-06-14 18:30:25

Facilitando significativamente la lectura de los registros.

Integración en el Proyecto

La función “registro-de-estados” utiliza:

```
(local-time:now)  
(local-time:format-timestring)
```

...para generar marcas temporales legibles.

De esta forma se cumple el requisito de mostrar fechas y horas comprensibles para el operador del sistema.



Bitácora de Depuración

Durante el desarrollo surgieron diversos errores que permitieron comprender mejor el funcionamiento de Common Lisp.

- **Error 1 - Número incorrecto de argumentos**

Valor ingresado → (transicion 'en-rojo)

Error obtenido → invalid number of arguments

Causa: La función requiere dos argumentos y sólo se proporcionó uno.

Solución: Verificar la cantidad de parámetros antes de ejecutar la función.

- **Error 2 — Variable sin valor**

Valor ingresado → (timer hola)

Error obtenido → variable HOLA has no value

Causa: Se intentó evaluar un símbolo sin definir.

Solución: Utilizar valores numéricos válidos para representar timestamps.

- **Error 3 — División por cero**

Error producido durante pruebas de cálculo porcentual.

Causa: La duración total del ciclo resultó igual a cero.

Solución: Validar previamente los valores utilizados en los cálculos.

- **Error 4 — Librería no cargada**

Error producido al utilizar funciones de Local-Time sin haber cargado previamente la librería.

Causa: Quicklisp o Local-Time no se encontraban correctamente inicializados.

Solución: Instalar y cargar la librería antes de ejecutar el sistema.



Fase 3 — Estudio Comparativo: Common Lisp vs. Clojure

Lenguaje asignado: “Clojure”

Código fuente de las funciones reimplementadas: “comparativa/solucion.clj”.

Presentación breve del lenguaje

Clojure es un lenguaje de programación funcional perteneciente a la familia Lisp, creado por Rich Hickey en el año 2007. Se ejecuta principalmente sobre la Máquina Virtual de Java (JVM), lo que le permite aprovechar el ecosistema de bibliotecas y herramientas desarrolladas para Java.

Al igual que Common Lisp, utiliza una sintaxis basada en listas y expresiones entre paréntesis. Sin embargo, incorpora características modernas orientadas a la programación funcional, especialmente el uso de estructuras de datos inmutables y mecanismos seguros para el manejo de concurrencia.

Entre sus principales características se destacan:

- Inmutabilidad por defecto.
- Programación funcional.
- Interoperabilidad con Java.
- Uso de keywords para representar estados y etiquetas.
- Soporte para programación concurrente.

Industrias y áreas donde se utiliza

Clojure es utilizado principalmente en:

- Sistemas backend.
- Plataformas financieras.
- Procesamiento de datos.
- Aplicaciones web.
- Sistemas distribuidos.
-

Empresas que utilizan Clojure

Algunas empresas conocidas que han utilizado o utilizan Clojure son:

- Nubank.
- Walmart.
- Netflix.
- Cisco.
- Atlassian.



Funciones reimplementadas

La consigna de la Fase 3 solicitó reimplementar únicamente las funciones “transición” y “timer” utilizando el lenguaje asignado.

Para ello se tomó como referencia la Iteración 2 desarrollada en Common Lisp, donde se incorpora el estado intermitente dentro del ciclo semafórico.

Las principales diferencias observadas entre ambas implementaciones fueron:

- En Common Lisp se utilizan símbolos cotizados como 'en-rojo.
- En Clojure se utilizan keywords como :en-rojo.
- Los resultados se representan mediante vectores inmutables.
- Se mantiene la misma lógica funcional utilizada en la versión original.

La funcionalidad general del sistema permanece idéntica en ambos lenguajes.

Respuestas a las preguntas teóricas

Pregunta 1 — Ventajas de los Keywords frente a los símbolos cotizados:

En Common Lisp los estados suelen representarse mediante símbolos cotizados, por ejemplo:

➔ 'en-rojo

En Clojure la forma más habitual de representar estados es mediante keywords:

➔ :en-rojo

Las ventajas principales son:

- No necesitan comillas para ser utilizadas.
- Son fáciles de leer y reconocer.
- Se utilizan habitualmente para representar estados y etiquetas.
- Permiten realizar búsquedas sencillas dentro de mapas.
- Su comparación es muy eficiente.

Por estas razones resultan especialmente adecuadas para modelar los estados del semáforo.

Pregunta 2 — Cómo maneja Clojure el paso del tiempo sin modificar variables:

Una de las características principales de Clojure es la inmutabilidad de sus estructuras de datos.

En lugar de modificar variables durante la ejecución, el estado se transmite mediante argumentos y valores de retorno.



En nuestro caso, la función “timer” recibe un timestamp y devuelve el estado correspondiente del semáforo. Cada llamada produce un resultado nuevo sin modificar ningún dato existente.

Para representar una secuencia temporal pueden utilizarse mecanismos como:

- Recursión de cola mediante loop y recur.
- Funciones de orden superior como map, mapv o reduce.

De esta manera el paso del tiempo se modela como una sucesión de valores inmutables, respetando los principios de la programación funcional.

Conclusión del grupo

La experiencia de trabajar con Clojure resultó interesante debido a su cercanía con Common Lisp.

La sintaxis general es similar, lo que facilitó la comprensión inicial del lenguaje. Sin embargo, Clojure incorpora conceptos modernos relacionados con la inmutabilidad y el manejo seguro del estado, aspectos que se encuentran muy presentes en el paradigma funcional.

Durante el desarrollo observamos que muchas de las ideas aplicadas en la implementación original en Common Lisp pueden trasladarse fácilmente a Clojure, aunque utilizando herramientas y convenciones propias del lenguaje.

La utilización de keywords para representar estados, las estructuras de datos inmutables y las herramientas para programación concurrente fueron algunos de los aspectos más destacados.

Como conclusión general, consideramos que Clojure constituye una alternativa moderna dentro de la familia Lisp y permite desarrollar aplicaciones robustas manteniendo los principios fundamentales de la programación funcional.

Bibliografía

- Sitio oficial de Clojure: <https://clojure.org>
- Documentación oficial de estructuras de datos: https://clojure.org/reference/data_structures
- Documentación oficial de special forms: https://clojure.org/reference/special_forms
- REPL utilizado para pruebas: <https://tryclojure.org>
- Información sobre Clojure en la industria: <https://www.cognitect.com>
- Información institucional de Nubank: <https://nubank.com.br>